

Проверка гипотезы о равенстве математических ожиданий

Вариант 5

1. Постановка задачи

Даны две независимые выборки $X_1, \dots, X_m$ и $Y_1, \dots, Y_n$ из нормальных генеральных совокупностей:

$$X_i \sim N(\mu_1, \sigma^2), \quad Y_j \sim N(\mu_2, \sigma^2).$$

Дисперсии неизвестны, но считаются равными. Объемы выборок: $m = 50$, $n = 54$. Уровень значимости: $\alpha = 0.05$.

Требуется проверить гипотезу:

$$H_0 : \mu_1 = \mu_2 \quad \text{против} \quad H_1 : \mu_1 \neq \mu_2$$

с помощью двустороннего критерия Стьюдента с объединенной оценкой дисперсии.

2. Теоретическое обоснование

2.1. Выборочные характеристики

Выборочные средние и исправленные дисперсии задаются формулами:

$$\bar{X} = \frac{1}{m} \sum_{i=1}^m X_i, \quad \bar{Y} = \frac{1}{n} \sum_{j=1}^n Y_j,$$
$$s_X^2 = \frac{1}{m-1} \sum_{i=1}^m (X_i - \bar{X})^2, \quad s_Y^2 = \frac{1}{n-1} \sum_{j=1}^n (Y_j - \bar{Y})^2.$$

2.2. Статистика критерия

При гипотезе H_0 объединенная оценка дисперсии равна:

$$s_p^2 = \frac{(m-1)s_X^2 + (n-1)s_Y^2}{m+n-2}.$$

Стандартная ошибка разности средних:

$$s_e = \sqrt{s_p^2 \left(\frac{1}{m} + \frac{1}{n} \right)}.$$

Наблюдаемая статистика критерия:

$$t_{\text{obs}} = \frac{\bar{X} - \bar{Y}}{s_e}.$$

При верной H_0 статистика t_{obs} имеет распределение Стьюдента с числом степеней свободы

$$\nu = m + n - 2.$$

Критическая область двустороннего критерия:

$$|t_{\text{obs}}| > t_{1-\frac{\alpha}{2}, \nu}.$$

3. Расчеты по данным

По выборкам получены оценки:

- $\bar{X} = 33.760000$,
- $\bar{Y} = 34.722222$,
- $s_X^2 = 23.737143$,
- $s_Y^2 = 26.996855$.

Отсюда:

- $s_p^2 = 25.430915$,
- $\nu = 102$,
- $t_{\text{obs}} = -0.972209$,
- $t_{\text{crit}} = \pm 1.983495$,
- $p\text{-value} = 0.333247$.

95%-й доверительный интервал для разности математических ожиданий:

$$(\mu_1 - \mu_2) \in (-2.925343; 1.000898).$$

4. Принятие решения

Поскольку

$$|t_{\text{obs}}| = 0.972209 < 1.983495 = t_{\text{crit}},$$

наблюдаемое значение статистики не попадает в критическую область.

Также

$$p\text{-value} = 0.333247 > 0.05,$$

следовательно, нет оснований отвергать нулевую гипотезу H_0 .

5. Вывод

На уровне значимости $\alpha = 0.05$ статистически значимых различий между математическими ожиданиями двух генеральных совокупностей не обнаружено.

Итоговый вывод: **нет оснований утверждать, что μ_1 и μ_2 различаются.**

6. Приложение: исходный код

```
from __future__ import annotations
```

```

import argparse
from pathlib import Path

import numpy as np
from scipy import stats

def read_sample(path: Path) -> np.ndarray:
    """Читает числовые значения из последнего столбца CSV, пропуская
    заголовок."""
    data = np.genfromtxt(path, delimiter=";", skip_header=1, usecols=-1,
dtype=float)
    if np.isscalar(data):
        data = np.array([float(data)], dtype=float)
    data = np.asarray(data, dtype=float)
    data = data[~np.isnan(data)]
    if data.size == 0:
        raise ValueError(f"В файле {path} не найдено числовых данных")
    return data

def pooled_t_test(sample_x: np.ndarray, sample_y: np.ndarray, alpha: float =
0.05) -> dict[str, float | int | bool]:
    """Двухвыборочный t-тест Стьюдента при равных (объединенных)
    дисперсиях."""
    # Размеры выборок.
    m = sample_x.size
    n = sample_y.size

    # Выборочные средние.
    mean_x = float(np.mean(sample_x))
    mean_y = float(np.mean(sample_y))
    # Несмещенные выборочные дисперсии (ddof=1 -> деление на n-1).
    var_x = float(np.var(sample_x, ddof=1))
    var_y = float(np.var(sample_y, ddof=1))

    # Объединенная оценка дисперсии при предположении, что  $\sigma_x^2 = \sigma_y^2$ .
    pooled_var = ((m - 1) * var_x + (n - 1) * var_y) / (m + n - 2)
    # Стандартная ошибка разности средних.
    se_diff = float(np.sqrt(pooled_var * (1.0 / m + 1.0 / n)))

    # Готовый двусторонний t-тест из SciPy для статистики и p-value.
    test = stats.ttest_ind(sample_x, sample_y, equal_var=True,
alternative="two-sided")
    t_stat = float(test.statistic)
    p_value = float(test.pvalue)

    # Число степеней свободы и критическое значение t для уровня значимости
    alpha.
    df = m + n - 2
    critical = float(stats.t.ppf(1.0 - alpha / 2.0, df))

```

```

# 95% доверительный интервал для разности матожиданий ( $\mu_x - \mu_y$ ).
diff = mean_x - mean_y
ci_low = diff - critical * se_diff
ci_high = diff + critical * se_diff

return {
    "m": m,
    "n": n,
    "mean_x": mean_x,
    "mean_y": mean_y,
    "var_x": var_x,
    "var_y": var_y,
    "pooled_var": pooled_var,
    "t_stat": t_stat,
    "critical": critical,
    "p_value": p_value,
    "ci_low": ci_low,
    "ci_high": ci_high,
    # Если |t| больше критического, нулевая гипотеза отклоняется.
    "reject_h0": abs(t_stat) > critical,
}

def format_result(result: dict[str, float | int | bool], alpha: float) -> str:
    decision = "Отклоняем H0" if bool(result["reject_h0"]) else "Нет оснований
отклонять H0"
    lines = [
        "Двухвыборочный t-тест Стьюдента (равные дисперсии)",
        f"alpha = {alpha:.2f}",
        f"m = {result['m']}, n = {result['n']}",
        f"x_mean = {float(result['mean_x']):.6f}, y_mean =
{float(result['mean_y']):.6f}",
        f"s2_x = {float(result['var_x']):.6f}, s2_y =
{float(result['var_y']):.6f}",
        f"pooled_variance = {float(result['pooled_var']):.6f}",
        f"t = {float(result['t_stat']):.6f}",
        f"t_crit = +/-{float(result['critical']):.6f}",
        f"p_value = {float(result['p_value']):.6f}",
        f"95% ДИ для ( $\mu_x - \mu_y$ ): ({float(result['ci_low']):.6f},
{float(result['ci_high']):.6f})",
        f"Решение: {decision}",
    ]
    return "\n".join(lines)

def main() -> None:
    parser = argparse.ArgumentParser(description="Двухвыборочный t-тест
Стьюдента с NumPy и SciPy")
    base_dir = Path(__file__).resolve().parent
    parser.add_argument("--x", type=Path, default=base_dir /
"variant_5_sample_X.csv", help="Путь к CSV первой выборки")

```

```
    parser.add_argument("--y", type=Path, default=base_dir /
"variant_5_sample_Y.csv", help="Путь к CSV второй выборки")
    parser.add_argument("--alpha", type=float, default=0.05, help="Уровень
значимости")
    args = parser.parse_args()

    sample_x = read_sample(args.x)
    sample_y = read_sample(args.y)
    result = pooled_t_test(sample_x, sample_y, alpha=args.alpha)
    print(format_result(result, args.alpha))

if __name__ == "__main__":
    main()
```